



© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2023

M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_16

16. Tooling and Frameworks

Matt Frisbie¹

(1) California, CA, USA

Modern browser extensions are rarely written from scratch. There is a wealth of developer tools at your disposal that allow you to efficiently build and test extensions in your chosen JavaScript framework and seamlessly publish them to multiple extension marketplaces.

Building Extensions with React

For developers who wish to build a browser extension with a JavaScript framework, the most common choice is React. How you ultimately fold React into your extension will largely depend on the scope and complexity of your application. Some things to consider:

- **Does your extension have one or many entrypoints?** Here, “entrypoint” can mean either an HTML file that will be loaded (popup.html or options.html), or a UI rendered via a content script. A single HTML entrypoint will greatly simplify the architecture of your extension.
- **Does your extension UI need to share state between different views?** Extensions only have an asynchronous shared storage mechanism, which makes using tools like Redux more complicated.
- **Does your extension require a React UI inside a content script?** Mounting a single page application in the page is different than rendering it inside a controlled HTML file.

- **How will your single page application use routing?** Content scripts, options pages, and popup pages all have different routing considerations that may affect how you roll out your React app.

Single Entrypoint React Extensions

If your extension only requires a single entrypoint (such as popup or options), then reaching for an app generated by `create-react-app` (CRA) can be a good option. In this section, we'll discuss how to create a very simple extension with CRA, and we'll also cover some of the drawbacks with using this strategy. Let's begin by generating a CRA application with TypeScript:

```
$ npx create-react-app mvx-react --template typescript
```

This will churn out a basic application structure that is designed for the web. We will need to customize it to make it usable in an extension. Begin by adding the type definitions for the WebExtensions API:

```
$ yarn add @types/chrome -D
```

The CRA generates a `manifest.json` intended for a progressive web application. It's already in the `public` directory, so it's perfect for you to repurpose for an extension manifest. Update the file with the following:

manifest.json

```
{
  "name": "MVX React",
  "description": "Minimum Viable Extension - React",
  "version": "0.0.1",
  "manifest_version": 3,
  "action": {
    "default_popup": "index.html"
  },
  "icons": {
    "16": "logo192.png",
```

```
"48": "logo192.png",  
"128": "logo192.png"  
}  
}
```

One final change is a minor CSS tweak. Since this is rendering inside the popup container, we need to enforce minimum dimensions – otherwise the browser will collapse the container to the size of the content. Add the following rules to `index.css`:

index.css

```
body {  
  ...  
  width: 400px;  
  min-height: 400px;  
}
```

This isn't ready to run yet. When building your application, CRA will try to place JavaScript inline in the page. Manifest v3 extensions explicitly disallow this, so you will need to configure the CRA build process to only load scripts from a URL. You can prevent CRA from inlining scripts by setting the environment variable `INLINE_RUNTIME_CHUNK` to `false`. There's a handful of ways to do this, but the simplest is to just prefix the assignment to the `package.json` commands:

package.json

```
{  
  "scripts": {  
    "start": "INLINE_RUNTIME_CHUNK=false react-scripts start",  
    "build": "INLINE_RUNTIME_CHUNK=false react-scripts build",  
    "test": "react-scripts test",  
    "eject": "react-scripts eject"  
  },  
  ...  
}
```

```
}
```

The app is ready to be installed as an extension. If we were running this as a website, we would execute `npm run start` to boot up a development server. However, because manifest v3 forbids loading code from remote sources – specifically, a server running on `localhost:3000` – this will not work. Instead, you will need to execute `npm run build` and load the build directory into the browser. When the build succeeds, load the extension into the browser and open the popup. You should see the CRA default page (Figure 16-1).

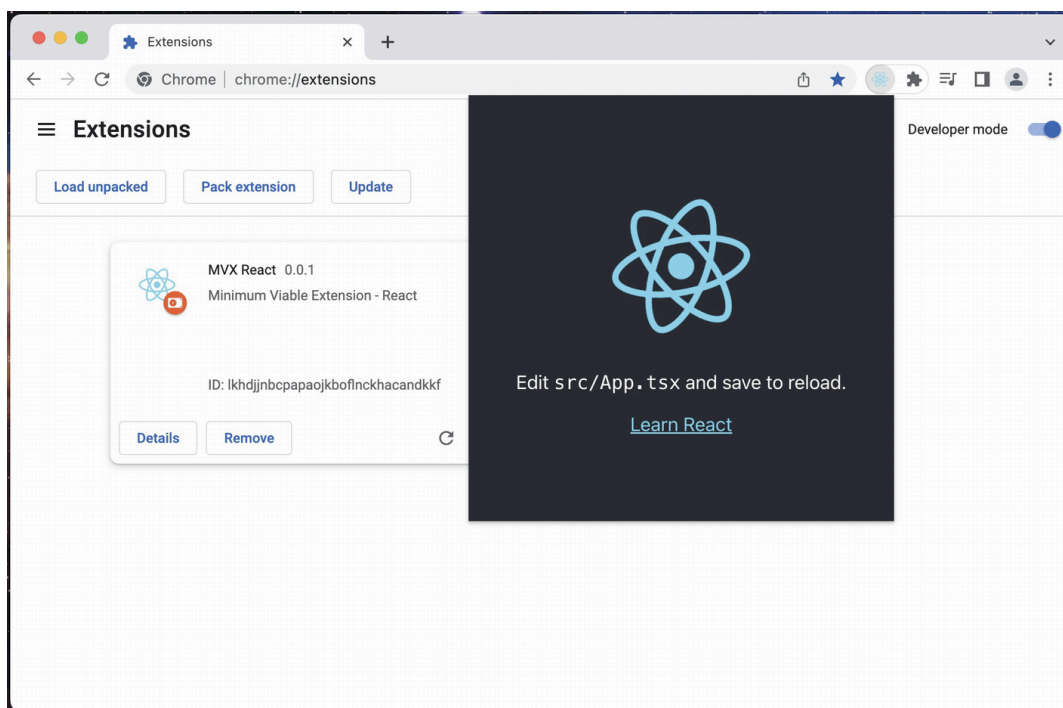


Figure 16-1 The Create React App index.html rendered in a popup

Tip Using `npm run build` is a suitable workaround for smaller projects, but it is not a scalable solution. Read on in the chapter for better build tools.

Multiple Entrypoint React Extensions

If your app has more than one entrypoint, then `create-react-app` is probably not a good solution for you. You will need a development configuration that supports multiple independent entrypoints, includ-

ing independent JS and CSS bundles. Read on in the chapter for several excellent solutions.

Reactive State Management

When managing state for a complex single page application, most developers will reach for a reactive state management container such as Redux. It is often advantageous to persist and rehydrate state, allowing for the application to remain consistent across reloads. There are two primary challenges in the context of browser extensions:

- The typical storage APIs that we would use such as `localStorage` or `IndexedDB` are not shared between extension pieces. For example, a popup writing to `localStorage` will be isolated from a content script writing to `localStorage` because their scripts are running on different origins!
- The only shared storage mechanism, `chrome.storage`, is asynchronous.

Often, you may only need to share a small piece of state, such as authentication data, between multiple extension views. Things like route state, server-loaded data, and other view-centric information are likely bound to a specific extension view, and therefore persisting it in something like `localStorage` may be acceptable. For example, a popup that loads a user's preferences from the server is safe to store that data locally if it will not be displayed anywhere else.

In situations where state must be shared between components, it is possible to configure Redux to use `chrome.storage` as an asynchronous store. The `chrome.storage.onChanged` event means that each view can react to other views mutating the store. There are two popular GitHub repositories that implement this:

- <https://github.com/ssorallen/redux-persist-webextension-storage>

- <https://github.com/robinmalburn/redux-persist-chrome-storage>

Tip If you're not familiar with Redux, read up on the React Redux library here: <https://react-redux.js.org/>.

Routing

When managing view state for a complex single page application, most developers will reach for a routing solution such as React Router. There are two primary challenges in the context of browser extensions:

- In views such as popups and options pages, the browser is loading an HTML file directly, so using an out-of-the-box routing solution will lead to invalid paths such as `index.html/foo/bar` that will break on reload.
- In views rendered via content scripts, you should not modify the host page URL whatsoever. Assume the host page will use the URL hash or query string fully, which can break any routing values you append to it.

Fortunately, both of these problems have straightforward solutions.

Extension Views and HashRouter

Across all browsers, extension views have a URL structure similar to the following:

```
extension-protocol://path/to/file.html
```

All major single page application routers support some form of URL hash routing, which will not interfere with the extension path and can survive page reloads:

```
extension-protocol://path/to/file.html#/your/app/route
```

For example, React Router can implement this routing strategy using `HashRouter`.

Tip Documentation on `HashRouter` can be found here:

<https://v5.reactrouter.com/web/api/HashRouter>.

If you want the popup to always open the most recent URL, you can dynamically update your popup URL via `chrome.action.setPopup()` and include the hash route.

Content Scripts and `MemoryRouter`

For views that render via a content script, you may have a complicated user interface that can benefit from routing, but modifying the page URL bar is not an option. All major single page application routers support some form of in-memory routing. These routers are often intended for native applications that do not have a URL, but they work equally well for extension content script views. React Router can implement this routing strategy using `MemoryRouter`.

If you need your router state to survive page reloads, you can persist your router state in your app state. Be aware that you may need to account for multiple route states across different tabs.

Tip Documentation on `MemoryRouter` can be found here:

<https://v5.reactrouter.com/web/api/MemoryRouter>

Mozilla Tools

Mozilla maintains a suite of tools geared for developing browser extensions. To a certain extent they are geared toward developing for Firefox, but in general they can be effectively used to develop for any browser.

web-ext

```
https://github.com/mozilla/web-ext
```

The `web-ext` project is a suite of CLI tools for developing and publishing browser extensions. It features these commands:

- `web-ext build` creates an extension package from source
- `web-ext sign` signs the extension so it can be installed in Firefox
- `web-ext run` runs the extension
- `web-ext lint` validates the extension source
- `web-ext docs` opens the web-ext documentation in a browser

There is also a Webpack plugin that wraps `web-ext` (not maintained by Mozilla):

```
https://github.com/hiikezoe/web-ext-webpack-plugin
```

Running `webpack` by itself will build the extension, effectively running `web-ext build` on the output of your Webpack build.

webextension-polyfill

```
https://github.com/mozilla/webextension-polyfill
```

Browsers are gradually moving the WebExtensions API over to an `async/await` friendly format, but the transition is not yet complete. Mozilla maintains this polyfill library to make the entire API return promises, eliminating the need for callbacks.

Bundlers and CLI Tools

When developing browser extensions, I strongly recommend using some form of software to assist in building and packaging the exten-

sion – especially if you are using a JavaScript framework like React, or have an extension with multiple entrypoints. Because the extension and framework space is currently undergoing a seismic shift, there are a lot of tools and repositories out there that are outdated: either they target only manifest v2, or don't support newer versions of frameworks. In this selection, I've hand-selected some projects that are actively maintained and can support development of browser extensions.

Warning A major feature of bundlers is Hot Module Replacement (HMR), the ability to hot swap out portions of an application as the developer updates the source files. For manifest v3 extension development, due to script restrictions, this is strictly incompatible; you will need to disable conventional HMR. Some bundlers rework HMR by altering it to programmatically reload the extension.

Parcel

<https://parceljs.org/>

If you are choosing an open-source tool for browser extension development, Parcel is by far my top choice. It has excellent support for manifest v3, multiple entrypoints, TypeScript, React, Vue, and Sass.

For developers coming from the world of Webpack, Parcel may be a bit disorienting at first. Whereas Webpack uses a dense configuration file to organize how it will build an application, Parcel is configuration-free. In lieu of a verbose configuration file, it uses pre-configured NPM packages that enable Parcel to implicitly understand how to build and bundle an application.

You can install the `@parcel/config-webextension` NPM package (<https://parceljs.org/recipes/web-extension/>) to enable Parcel to build your web extension. Using this recipe, Parcel can parse your `manifest.json` file for entrypoints. It will then parse those HTML files

to understand how it should build the JS and CSS bundles for each separate entrypoint.

Translation from uncompiled file extensions such as TypeScript (.ts), React (.jsx/.tsx), Vue (.vue), and Sass (.scss) files is automatic, meaning you can directly use these files in the manifest and HTML entrypoints and Parcel will handle the compilation and file path replacement for you. Some examples:

manifest.json

```
{
  ...
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "js": ["content-script.tsx"],
      "css": ["content-script.scss"]
    }
  ],
  ...
}
```

popup.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link href="popup.scss" rel="stylesheet" />
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="popup.tsx"></script>
  </body>
</html>
```

Note Refer to the Parcel documentation for instructions on how to set up a new browser extension project, as well as coverage of other features: <https://parceljs.org/recipes/web-extension/>.

Webpack

<https://webpack.js.org/>

Webpack is often the go-to choice for website development, and it certainly continues to be a sensible choice for browser extension development. Unlike Parcel, Webpack favors explicitly defining behavior inside a config file. For particularly complicated extensions, it may be advantageous for you to have the ability to explicitly dictate how Webpack should compile and bundle your extension files. Webpack also has a significantly larger community behind it, which may be an important consideration for long-term support.

For developers who wish to begin with a Webpack config file geared for building manifest v3 browser extensions, I would recommend one of the following two repositories:

- <https://github.com/lxieyang/chrome-extension-boilerplate-react>
- <https://github.com/sszczep/chrome-extension-webpack>

Warning There is a relatively large number of GitHub repositories containing outdated Webpack browser extension templates. These target older versions of Webpack or are strictly limited to manifest v2. Be careful when choosing a starter repository, it can often be very painful to move off of it.

Plasmo

<https://www.plasmo.com/>

Plasmo's website says it best:

Plasmo is a platform for developing browser extensions.

Shipping extensions like shipping websites – FAST.

There are many aspects of the browser extension development and deployment process that are painful, and Plasmo is designed to hide all of the unpleasantness under the hood. As you will see in this section, it has a robust set of features that hide all the nastiness of generating multiple manifest versions, supporting different browsers, and deploying to all the extension marketplaces.

Tip For its ease of use, engaged maintainers, active developer community, and powerful featureset, I would recommend using Plasmo over anything else to develop a browser extension.

High-level Overview

The Plasmo framework builds opinionated abstractions on top of Parcel that considerably improves the extension development experience. Like Parcel, Plasmo features a convention-over-configuration design philosophy. For example, if you want to add an options page to your extension, you can just create an `options.tsx` React component in the project's root directory, and Plasmo will do the rest: automatically compile that file into JS/CSS/HTML assets, generate a new `manifest.json` with the new HTML file as the options page, and reload the extension.

Tip Plasmo is incredibly easy to start using right away. Check out their *Getting Started* guide here: <https://docs.plasmo.com/#getting-started>

JavaScript Frameworks

Plasmo is unopinionated when it comes to JavaScript frameworks. It supports React, Vue, and Svelte. By default, the CLI `init` command will generate a React application, but switching over to a different framework is trivial:

- To use Vue, install the `vue` package and change your files to use `.vue` extensions (e.g., `popup.vue`).
- To use Svelte, install `svelte` and `svelte-preprocess`, initialize your `svelte.config.js` file, and change your files to use `.svelte` extensions (e.g., `popup.svelte`).

Documentation and Examples

Bad documentation has long been a thorn in the side of extension development. Manifest versions are inconsistent, API coverage is spotty or unclear, and examples are inconsistent. Plasmo features *very* robust and complete documentation:

```
https://docs.plasmo.com/
```

Plasmo's docs also include a collection of "quickstarts," which are detailed instructions on how to incorporate popular libraries. This is especially useful for developers like me, because the first two things I do in a new project is install and set up Tailwind and Redux.

```
https://docs.plasmo.com/quickstarts
```

For developers who prefer to learn by example, the Plasmo GitHub account features a wide collection of examples explaining how to accomplish common tasks:

```
https://github.com/PlasmoHQ/examples
```

Differential Build Outputs

Plasmo has some very useful CLI features for generating application builds. It is capable of generating both manifest v2 and manifest v3 outputs with a command line flag, like so:

```
# Generates build/chrome-dev-mv3/ directory
```

```
pnpm dev --target=chrome-mv3
# Generates build/firefox-dev-mv2/ directory
pnpm dev --target=firefox-mv2
# Generates build/chrome-prod-mv3/ directory
pnpm build --target=chrome-mv3
# Generates build/firefox-prod-mv2/ directory
pnpm build --target=firefox-mv2
```

Notechrome-mv3 is the default target

Furthermore, Plasmo can create a zip of these builds that can be directly handed off to the extension marketplaces. This is done with the `--zip` flag:

```
# Generates build/chrome-prod-mv3.zip
pnpm build --zip
# Generates build/firefox-prod-mv2.zip
pnpm build --target=firefox-mv2 --zip
```

Automatic Manifest Generation

Instead of writing a `manifest.json` file yourself, Plasmo generates the manifest based on your source files and configurations you export from your code – similar to how Next.js abstracts page routing and SSG with the file system and page components.

Values in your project's `package.json` will typically be redundant with the corresponding values in your extension manifest. Plasmo will automatically copy over values from `package.json` into your `manifest.json`:

- `packageJson.version` -> `manifest.version`
- `packageJson.displayName` -> `manifest.name`
- `packageJson.description` -> `manifest.description`
- `packageJson.author` -> `manifest.author`
- `packageJson.homepage` -> `manifest.homepage_url`

Icon Generation

Browser extensions display its icon throughout the browser, and the icons must be tediously listed in multiple places inside the manifest. As shown below, Plasmo will take a single source 512x512 icon in the `assets/` directory, generate multiple resized icons, and automatically include them in the manifest wherever needed (Figure 16-2).

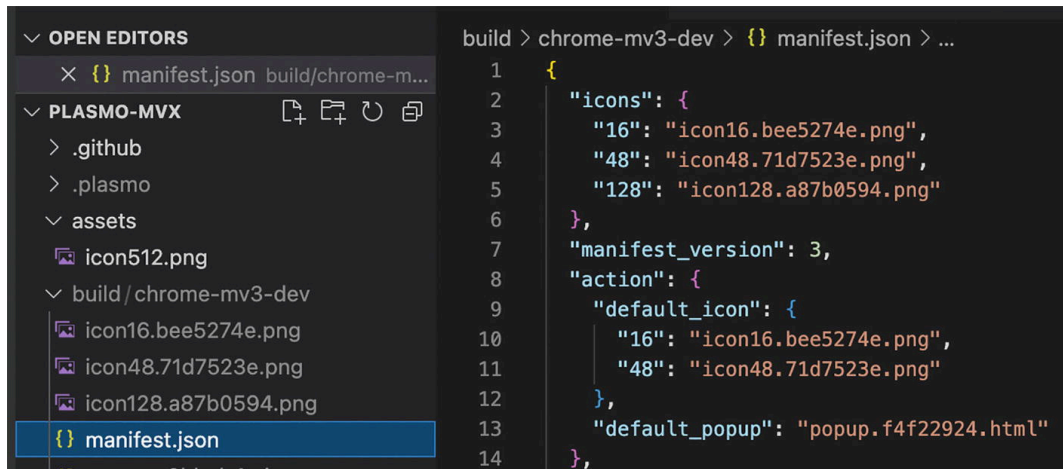


Figure 16-2 Dynamic icon generation from single asset image

Bundling Remote Code

In manifest v3, loading and executing remote code is strictly forbidden. To solve this problem, Plasmo will parse your `import` statements for remotely hosted assets, fetch them, and bundle them inside the extension. In the following screenshot, we can see that `popup.tsx` is importing the remote Google Tag Manager JavaScript, and the file is automatically included in the `build/chrome-mv3-dev/` directory (Figure 16-3).

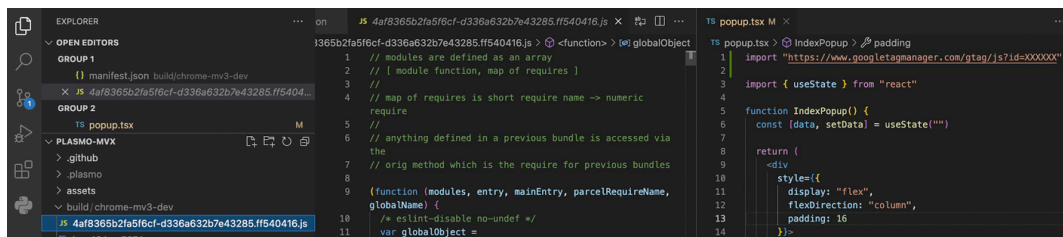


Figure 16-3 Plasmo automatically bundles the gtag.js into the extension

Environment Variables

Plasmo features an intuitive solution for environment variables based on the `dotenv` NPM package. Environment variables can be placed in a `.env` file in

the root directory of your project, and they will become available:

- In JavaScript via the `process.env` namespace
- In `import` URLs
- In `package.json` manifest overrides

The following snippets demonstrate each of these:

.env

```
PLASMO_PUBLIC_FOO=foo
PLASMO_PUBLIC_GTAG_ID=123456789
PLASMO_PUBLIC_CRX_PUBLIC_KEY=asdf-1234-asdf-1234
```

popup.tsx

```
import "https://www.googletagmanager.com/gtag/js?id=$PLASMO_PUBLIC_GTAG_ID"
function IndexPopup() {
  return <div>{process.env.PLASMO_PUBLIC_FOO}</div>;
}
export default IndexPopup
```

manifest.json

```
{
  "key": "$CRX_PUBLIC_KEY"
}
```

Note Plasmo mirrors the Next.js `.env` file convention with `.local`, `.production`, and `.development` suffix rules. Read more here: <https://docs.plasmo.com/workflows/env>.

Content Script Mounting

Injecting a user interface in a content script usually requires some bootstrapping, and Plasmo does all this for you as a Content Script UI.

You can define your component in a `content.*` file with a `PlasmoContentScript` definition. Plasmo will automatically add a corresponding `content_scripts` entry in your manifest. The following is an example of this:

content.tsx

```
import type { PlasmoContentScript } from "plasmo"
const config: PlasmoContentScript = {
  matches: ["<all_urls>"]
}
function IndexContentUI() {
  return (
    <h1
      style={{
        backgroundColor: "blue",
        color: "white",
        padding: "12rem"
      }}>
      I'm the content script widget!
    </h1>
  )
}
export default IndexContentUI
```

This widget will be injected on every web page. Note below that Plasmo is injecting the component inside a Shadow DOM container to prevent CSS interference (Figure [16-4](#)).

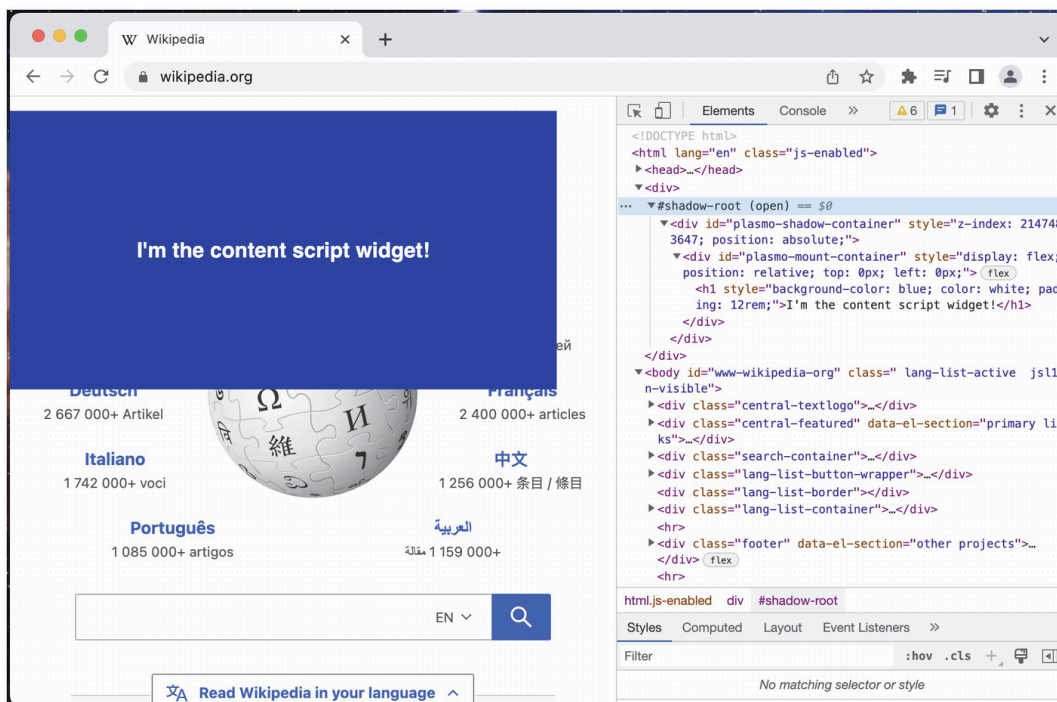


Figure 16-4 The injected content script widget inside a Shadow DOM

Extension-friendly Hot Module Replacement

Manifest v3 disallows traditional HMR, but Plasmio offers something similar. When developing your extension, Plasmio injects a web socket listener into the development build of the extension. Whenever a bundle change happens, Plasmio sends the refresh message and it either executes `chrome.runtime.reload()` or `location.reload()` depending on the context. While this isn't strictly the same as HMR, it still allows you to see code updates in real time.

Automated Deployment with Browser Platform Publish

Plasmio ships with a GitHub action called Browser Platform Publish (BPP) that will automatically publish your extension updates to all supported browser extension marketplaces. Setting up BPP involves the following steps:

1. **Perform the initial publish of your extension to each marketplace.** You'll need an existing extension on each marketplace before you can use BPP to publish updates.

2. **Generate credentials for each marketplace.** Extension marketplaces allow you to generate authentication tokens to automatically sign and publish updates. You will need to do this for each marketplace.
3. **Provide the credentials to BPP.** Once you've acquired your credentials for each marketplace, you'll provide it to the BPP GitHub action as an encrypted repository secret.

Once this one-time setup is performed, you will be able to seamlessly push updates to all the extension marketplaces simultaneously. The following links provide more details on context on how to set up and use BPP:

- Details on retrieving credentials:
<https://github.com/PlasmoHQ/bms/blob/main/tokens.md>
- GitHub action page for BPP:
<https://github.com/marketplace/actions/browser-platform-publisher>
- BPP GitHub repository: <https://github.com/PlasmoHQ/bpp>
- Introduction to using BPP:
<https://docs.plasmo.com/workflows/submit>

Useful Sites

- <https://buildingbrowserextensions.com> is the companion website of this book. It includes a link to the Browser Extension Explorer, a Chrome extension that consists of open source demos for each extension API.
- <https://developer.chrome.com/> is Chrome's official site to help you build Extensions, publish on the Chrome Web Store, optimize your website, and more.
- <https://extensionworkshop.com/> helps with creating and publishing Firefox add-ons that make browsing smarter, safer, and faster. You'll find the resources you need, whether you're getting started with extension development, preparing to launch your innovation, or developing a custom enterprise solution.

- <https://webext.eu/> quickly generates browser extension templates with just a few clicks. It is maintained by Mozilla. It is very handy for quickly prototyping extensions.

Summary

In this chapter, we discussed a wide variety of automated tools that you can use to speed up and simplify extension development. First, we discussed how React can be configured to neatly fit into an extension project, as well as how to integrate some popular React libraries. Next, we covered a variety of popular open source build tools that can be used to manage the inherent complexity of browser extensions. Finally, we covered all the different features that the Plasmo platform has to offer.